

Correction note

Prepended 21 September 2026 to the record as originally rendered, which follows unchanged.

Correction note — 21 September 2026 (D-42). The needs-review figure in this document, 168 of 430 (39.1%), was measured while a classifier defect was caching failed API batches as abstentions. How many of the 168 were affected cannot be recovered, because the cache does not timestamp its entries, so read the figure as an upper bound. At 442 rows the corrected figure is **128 (29.0%)**. Gate A's reconciliation (classified + uncertain = total) still holds: it checked arithmetic closure, not the classifier's output. The record below is left as written. See [docs/d42_failure_record.md](#) for the cause, the evidence and the fix.

Gate A — Mechanical Checkpoint

Phase 2 QA checkpoint, gate 1 of 4

Amr Shariff

Run 2 September 2026 (Day 15). Record compiled 8 September 2026 (Day 17).

[Purpose](#)

[What was run](#)

[Result](#)

[D-02 — the chat handle fix was missing from HEAD](#)

[Discovery](#)

[Why the suite did not catch it](#)

[The surviving bug](#)

[Fix \(P2.8e\)](#)

[D-05 — no request timeout on the Anthropic client](#)

[D-07 — injection predicates read the narration channel](#)

[A5b — the reconciliation](#)

[Defects logged, not fixed](#)

[Commit](#)

[What this gate bought](#)

Purpose

Gate A is the first of four gates in the Phase 2 QA checkpoint, run before P2.9a (synthetic demo data) and P2.9 (Community Cloud deployment).

Its scope is deliberately narrow: **the suite and all four live harnesses re-run on current HEAD, plus a data reconciliation**. Fast, objective, no judgement. The ordering rule was stated before the gate began — if Gate A is red, nothing below it means anything, because every judgement in Gate B is a judgement about code that Gate A has established is actually the code that ships.

One clause in the gate’s own definition did the heavy lifting:

Re-run on current HEAD. Day-14 terminal output does not count.

That clause was written to prevent a formality — recording yesterday’s numbers as today’s evidence. It caught a lost commit instead.

What was run

```
git --no-pager log --oneline -5
git status --short
python -m pytest tests\ -q
python -m tools.check_single_loader
python -m tools.check_uncertainty_honesty
python -m tools.check_summary_safety
python -m tools.check_chat_grounding
python -m tools.checkpoint_reconcile
```

The offline harnesses were run first and the two API-consuming harnesses separately, so the free checks could be read before spending on the paid ones.

Expected values were written down **before** running, from the Day-14 record: 223 passed, 6/6, 9/9, 11 summaries, 26/26. The gate stated in advance that any deviation was a finding, *including a higher number* — a higher number would mean uncommitted work unaccounted for.

Result

Check	Result	Verdict
A1 — full suite	229 passed (was 223)	PASS
A2 — check_single_loader	6/6	PASS
A3 — check_uncertainty_honesty	9/9	PASS
A4 — check_summary_safety	11 summaries	PASS
A5 — check_chat_grounding	26/26	PASS
A5b — reconciliation	262 + 168 = 430; 18 groups; 0 not-an-occupation	PASS

Gate A closed 2 September 2026, after remediation of three defects.

The gate did not pass on first run. The table above records the state after D-02, D-05 and D-07 were remediated and committed as P2.8e.

D-02 — the chat handle fix was missing from HEAD

Severity: high. Blocking.

Discovery

`tools/check_chat_grounding.py` would not run at all. It imported `parse_handle` from `src/assistant.py`, and the function did not exist.

Git archaeology showed the harness was last touched at `4e371c7` (P2.8b), which is *later* than the last commit to `assistant.py`. So the harness had been edited during P2.8b against an API that the committed `assistant.py` did not have.

The signature in HEAD settled it:

```
def _strip_indices(text: str, valid: set[int]) -> str:
```

`set[int]` — bare integer handles. That is the **pre-fix** signature. Day 14 had changed handles to prefixed strings (`"p87"`, not `87`) precisely because the sanitiser was corrupting prose. `parse_handle` is the function that turns `"p87"` back into `87`, and it was absent because the entire prefixed-handle change was absent.

Why the suite did not catch it

223 tests passed, which corroborated the diagnosis rather than contradicting it: `tests/test_assistant.py` was *also* the pre-fix version. Had the tests been updated for prefixed handles they would have failed against this `assistant.py`. Both files were lost together, which points at the `git reset --soft` and `--force-with-lease` sequence at the end of Day 14 as the mechanism.

Working tree was clean and git diff was empty, because what was on disk was what had been committed. Nothing observed the gap because the harness was never re-run before the session closed.

The surviving bug

`_strip_indices` scrubbed any bare digit that was also a valid `person_index`, so a real count in prose was replaced by “someone”. Its own docstring claimed `"3 people"` survived — but 3 is a valid index in a 430-row table, and it did not. On the live network, *“well over 30 in the table”* rendered as *“well over someone in the table”*.

Fix (P2.8e)

Handles are now `p`-prefixed strings via `make_handle` / `parse_handle`, so a count and a handle are distinguishable and `_strip_handles` can be unconditional — it no longer needs the table to decide what to strip.

Validation was split by layer rather than duplicated:

- `parse_answer` checks **shape** (a dict carrying an `i`)
- `resolve_people` checks **content** and owns `dropped_count`

Rejecting a malformed handle at parse time would make it vanish before the count could see it, and a silent drop is a quiet lie about completeness. A bare integer is dropped and counted, never coerced — a coerced reference is a guess wearing a resolution’s clothes.

Verified live: *“more than 30 people have data-related titles”* now survives intact.

D-05 — no request timeout on the Anthropic client

Severity: high.

A stalled response blocked indefinitely in `_receive_response_headers`. In the harness this reads as a hang. In the deployed app the chat panel would spin forever with no error and no degraded state.

That is the one failure mode `answer_question`’s *“never raises for an API problem”* contract did not cover — because it never returned at all.

Fix: 60-second timeout, 2 SDK retries.

D-07 — injection predicates read the narration channel

Severity: medium. Instrument defect.

Sections C2 and C4 of `check_chat_grounding` failed a model that had correctly resisted the injection.

The mechanism: rule 4 of the system prompt *requires* the model to note that it ignored a planted instruction. A predicate matching the injection’s own vocabulary in `ans.text` therefore detects the defence working and reports it as a failure.

C2 additionally matched “management” in any `why`, firing on “*Business finance management role*” — a correct `why` for a correctly classified person.

Fix: predicates now read the **claim channel** (`people`, `groups`, `unreviewed flags`, `dropped`) rather than the narration channel.

Consequence for the record: Day 14’s 26/26 was partly luck. It was measured against predicates that were sensitive to how verbosely the model narrated its own resistance. The 26/26 recorded at Gate A close means something different — it is measured against predicates that read what the model actually claimed.

A5b — the reconciliation

Gate A5 existed to settle a specific ambiguity. Two prior sessions had recorded “18” for the not-an-occupation count, but at least one of them may have been reading “*18 distinct classified hues*” — the number of SOC groups present in the data.

These are different quantities that happened to collide on the same number, which is exactly the kind of error that survives unchallenged into a README.

`tools/checkpoint_reconcile.py` reads the display table through the single loader with **no corrections applied**, so the output is base-table fact rather than session artefact.

Result: 262 classified + 168 uncertain = 430. 18 SOC groups present. 0 not-an-occupation nodes. The “18” was the group count. The not-an-occupation figure was zero.

Defects logged, not fixed

Per the checkpoint rule, nothing is fixed inside a gate unless it blocks the gate itself. These were named, phase-assigned, and left.

	Defect	Assigned
D-01	Harness invocation form wrong in README	P2.9
D-03	Intermediate iteration committed as final	process
D-04	Sweep the other Day-14 files for the same loss	P2.9
D-06	<code>.env</code> loading is implicit	P2.9
D-08	C2/C4 predicate labels misleading	Phase 4

Commit

Three defects, one commit, because they were found by one instrument pass and the mechanism is the valuable part of the record:

P2.8e: prefixed handles, request timeout, and honest injection predicates

The suite was re-run immediately before the push — “*precisely the step whose absence caused D-02.*”

Tests: **223 → 229**. Three cases that asserted bare-digit stripping as correct behaviour were rewritten; the live failure is now an explicit assertion.

What this gate bought

A checkpoint clause written to avoid a formality caught a **silently reverted commit that would otherwise have deployed at P2.9**.

The failure was invisible to every signal that normally indicates health:

- Working tree clean
- `git diff` empty
- 223 tests passing
- A recorded 26/26 from the previous session

Every one of those was true, and the shipped code was still wrong. What was missing was an **execution against current HEAD**. The harness had been edited to match an API that never got committed, and nothing ran it again before the session closed.

Two principles came out of this and have held since:

1. **Re-run the instruments on HEAD, not on yesterday's terminal.** A recorded number is evidence about a moment, not about a commit.
2. **Instrument integrity is a prerequisite for a verdict.** D-07 was a defect in the checker, not the product. A gate that trusts a broken instrument produces a verdict about nothing. This recurred at Gate B as D-20 and the lesson was already in the record when it did.